

Announcements

- Login, open an IDE, and Folio/browser for class
- Check grades in Folio
- Download recursion1.zip

1

Announcements

- Revel Exercises due
 - ~~Chap 17~~
- Project 9 due Friday, start of lab
 - See Program Submission requirements in Folio

2

Remaining Schedule

- Course evaluations
- Friday May 2 is last lab
- Mon May 5 is last class meeting
- Final day/time on class schedule/syllabus
- Do **NOT** use Gradescope as a calendar

3

Warm-Up Questions

- What are the advantages of using buffers?
- How do we use buffers when doing binary I/O?
- How do we know that buffered I/O is beneficial?
- How can we store and retrieve instances of objects in Java?
- How do random access files work?
- How can we implement this idea in Java?
- What considerations need to be made so this works?

4

Lecture Questions

- What is recursion?
- What is a recursive method?
- What are the benefits to using recursion?
- How can recursive methods be used for mathematical functions?
- How are recursive method calls handled in a call stack?
- How can problems be solved using recursion?

5

Chapter 18 – Recursion

CSCI 1302 – Programming Principles II

6

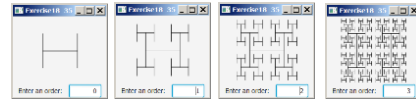
Motivations

- Suppose you want to find all the files in a directory that contains a particular word or phrase
- How do you solve this problem?
- An intuitive solution is to use **recursion** by searching the files in the subdirectories **recursively**

7

Motivations

- H-trees, depicted in Figure 18.1, are used in a very large-scale integration (VLSI) design as a clock distribution network for routing timing signals to all parts of a chip with equal propagation delays
- Displaying H-trees can be done with recursion



8

Recursion

- **Recursion** is a technique that leads to elegant solutions for problems that are difficult to solve with iteration
- A **recursive method** is a method that calls itself
 - Some iterative problems are more easily solved/expressed recursively

9

Math Functions

- A number of math functions are expressed using a **recurrence relation**
- Specifies a **base case (stopping point)** as well as a **general case** for all other values
- General case typically calls itself with a different value (**recursive call**)
 - Value often moves toward the base case

10

Factorial – Activity

- Find the Wikipedia page for factorial
- Locate the recurrence relation that defines factorial
- According to the page, what else is necessary for factorial when $n = 0$?

11

Computing Factorial

- $0! = 1$ Base case
- $n! = n * (n-1)!$ General case

- **Activity:** Write a method that takes an int parameter and recursively calculates and returns the factorial using the above recurrence relation (Test for 5!)
 - HINT: Blindly follow the formula

12

Computing Factorial

```

/** Return factorial for a specified number */
public static int factorial(int n) {
    if (n == 0) { // Base case
        return 1;
    } else {
        // Recursive call
        return n * factorial(n - 1);
    }
}

```

13

Computing Factorial

- Activity – Test this method using increasing values of n
- For what value of n do you start seeing issues?
 - What is happening?
 - Can you solve it easily?

14

Computing Factorial

```

/** Return factorial for a specified number */
public static long factorial(int n) {
    if (n == 0) { // Base case
        return 1;
    } else {
        // Recursive call
        return n * factorial(n - 1);
    }
}

```

15

Computing Factorial

- Activity – Test this method using an n of 4
- Chart out the number of calls necessary to return the factorial of 4
- How many times is factorial called to calculate this value?

16

Computing Factorial

factorial(4)

```

factorial(0) = 1;
factorial(n) = n*factorial(n-1);

```

17

Computing Factorial

factorial(4) = 4 * factorial(3)

```

factorial(0) = 1;
factorial(n) = n*factorial(n-1);

```

18

definition

Computing Factorial

```
factorial(4) = 4 * factorial(3)
             = 4 * 3 * factorial(2)
```

```
factorial(0) = 1;
factorial(n) = n * factorial(n-1);
```

19

definition

Computing Factorial

```
factorial(4) = 4 * factorial(3)
             = 4 * 3 * factorial(2)
             = 4 * 3 * (2 * factorial(1))
```

```
factorial(0) = 1;
factorial(n) = n * factorial(n-1);
```

20

definition

Computing Factorial

```
factorial(4) = 4 * factorial(3)
             = 4 * 3 * factorial(2)
             = 4 * 3 * (2 * factorial(1))
             = 4 * 3 * (2 * (1 * factorial(0)))
```

```
factorial(0) = 1;
factorial(n) = n * factorial(n-1);
```

21

definition

Computing Factorial

```
factorial(4) = 4 * factorial(3)
             = 4 * 3 * factorial(2)
             = 4 * 3 * (2 * factorial(1))
             = 4 * 3 * (2 * (1 * factorial(0)))
             = 4 * 3 * (2 * (1 * 1))
```

```
factorial(0) = 1;
factorial(n) = n * factorial(n-1);
```

22

definition

Computing Factorial

```
factorial(4) = 4 * factorial(3)
             = 4 * 3 * factorial(2)
             = 4 * 3 * (2 * factorial(1))
             = 4 * 3 * (2 * (1 * factorial(0)))
             = 4 * 3 * (2 * (1 * 1))
             = 4 * 3 * (2 * 1)
```

```
factorial(0) = 1;
factorial(n) = n * factorial(n-1);
```

23

definition

Computing Factorial

```
factorial(4) = 4 * factorial(3)
             = 4 * 3 * factorial(2)
             = 4 * 3 * (2 * factorial(1))
             = 4 * 3 * (2 * (1 * factorial(0)))
             = 4 * 3 * (2 * (1 * 1))
             = 4 * 3 * (2 * 1)
             = 4 * 3 * 2
```

```
factorial(0) = 1;
factorial(n) = n * factorial(n-1);
```

24

algorithm

Computing Factorial

```

factorial(0) = 1;
factorial(n) = n*factorial(n-1);

factorial(4) = 4 * factorial(3)
             = 4 * (3 * factorial(2))
             = 4 * (3 * (2 * factorial(1)))
             = 4 * (3 * (2 * (1 * factorial(0))))
             = 4 * (3 * (2 * (1 * 1)))
             = 4 * (3 * (2 * 1))
             = 4 * (3 * 2)
             = 4 * (6)

```

25

algorithm

Computing Factorial

```

factorial(0) = 1;
factorial(n) = n*factorial(n-1);

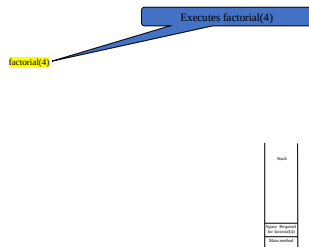
factorial(4) = 4 * factorial(3)
             = 4 * (3 * factorial(2))
             = 4 * (3 * (2 * factorial(1)))
             = 4 * (3 * (2 * (1 * factorial(0))))
             = 4 * (3 * (2 * (1 * 1)))
             = 4 * (3 * (2 * 1))
             = 4 * (3 * 2)
             = 4 * (6)
             = 24

```

26

algorithm

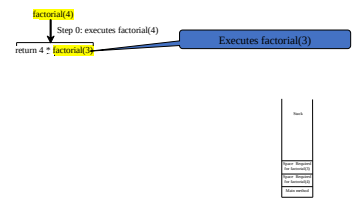
Trace Recursive factorial



27

algorithm

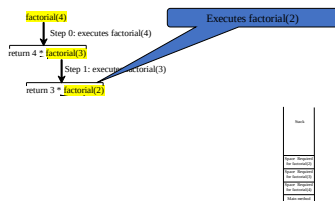
Trace Recursive factorial



28

algorithm

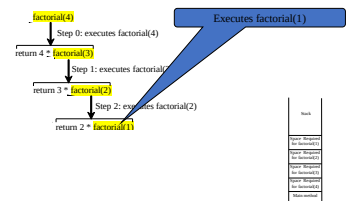
Trace Recursive factorial



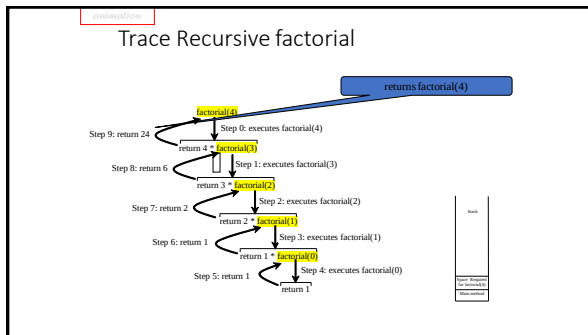
29

algorithm

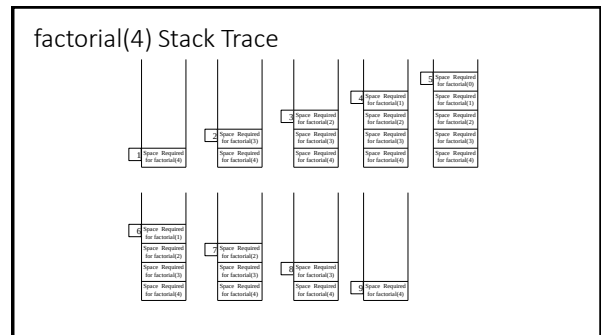
Trace Recursive factorial



30



37



38

Other Possible Recursive Examples

Try implementing at home:

$$f(0) = 0;$$

$$f(n) = n + f(n-1);$$

$$f(0) = 1;$$

$$f(n) = n - f(n-1);$$

39

Fibonacci – Activity

- Find the Wikipedia page for Fibonacci sequence (aka Fibonacci numbers)
- Determine the **recurrence relation** that defines Fibonacci
- What is the base case?
- What is the general case?

40

Fibonacci Numbers

Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55 89...

indices: 0 1 2 3 4 5 6 7 8 9 10 11

```

fib(3) = fib(2) + fib(1)
       = (fib(1) + fib(0)) + fib(1)
       = (1 + 0) + fib(1)
       = 1 + fib(1)
       = 1 + 1
       = 2
  
```

41

Fibonacci – Activity

- Based on the recurrence relation and the previous example, write a method named fib that takes an int parameter and returns the corresponding Fib number

Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55 89...

indices: 0 1 2 3 4 5 6 7 8 9 10 11

42

Fibonacci

```

/** Finding the Fibonacci number */
public static long fib(long index) {
    if (index == 0) // Base case
        return 0;
    else if (index == 1) // Base case
        return 1;
    else // Recursive calls
        return fib(index-1) + fib(index-2);
}

```

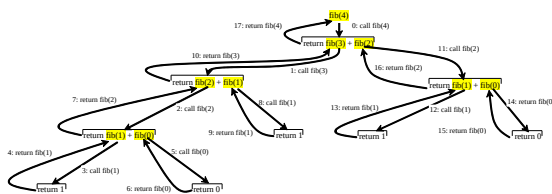
43

Fibonacci

- Test this method using an n of 4
- Chart out the number of calls necessary to return the Fibonacci number at index 4
- How many times is fib called to calculate this value?

44

Fibonacci Numbers, cont.



45

Characteristics of Recursion

- All recursive methods have the following characteristics:
 - One or more base cases (or stop cases) are used to stop recursion
 - Every recursive call reduces the original problem, bringing it closer to a base case until a base case is called
- To solve a problem using recursion, must be broken into subproblems
 - If subproblem resembles the original problem, can apply the same approach to solve the subproblem recursively
 - Subproblem is almost the same as the original problem in nature but with a smaller size

46

Problem Solving Using Recursion

- Start simple: how do you know if you are done consuming a drink? What is the first step?
 - Check if there is more to drink
 - If yes, drink
 - If no, stop
 - Repeat with remaining drink

47

Problem Solving Using Recursion

- Consider printing a message n times
- Two subproblems:
 - Print the message one time
 - Print the message for n-1 times
- The second problem is the same as the original problem with a smaller size
- Base case for the problem is $n == 0$
- So, problem can be solved using recursion

48

Problem Solving Using Recursion

- First, consider the method call:
`nPrintln("Welcome", 5);`
- Then, the method code:

```
public static void nPrintln(String msg, int n) {
    if (n >= 1) {
        System.out.println(msg);
        nPrintln(msg, n - 1);
    } // base case is n == 0, nothing to print
}
```

49

Palindrome – Iterative Solution

```
// Prompt the user to enter a string
System.out.print("Enter a string: ");
String s = input.nextLine();

int low = 0; // index of first character in the string
int high = s.length() - 1; // index of last character in the string
boolean isPalindrome = true;

while (low < high && isPalindrome) {
    if (s.charAt(low) != s.charAt(high)) {
        isPalindrome = false;
    }
    low++;
    high--;
}
```

50

Think Recursively

- Many problems in early chapters can be solved using recursion if you think recursively
- Recursive palindrome solution:

```
public static boolean isPalindrome(String s) {
    if (s.length() <= 1) // Base case
        return true;
    else if (s.charAt(0) != s.charAt(s.length()-1))
        return false; // Base case
    else
        return isPalindrome(s.substring(1, s.length()-1));
    // substring excludes endpoint, this is 1-length()-2
}
```

51

Activity

- Using the recursive palindrome code, test some Strings using the debugger
- What makes this code recursive?
- Is this solution more or less readable than the iterative version?
- Which mirrors the definition of a palindrome more closely?

52

Announcements

- Revel Exercises due
 - Chap 17
- Project 9 due Friday, start of lab
 - See Program Submission requirements in Folio

53

Remaining Schedule

- Course evaluations
- Friday May 2 is last lab
- Mon May 5 is last class meeting
- Final day/time on class schedule/syllabus
- Do **NOT** use Gradescope as a calendar

54